Student Marks Management System — Viva Report

Student Marks Management System — Viva Report

Module: iCD122 / iCD125 — Java GUI Coursework **Project:** StMangSys (Apache NetBeans, Ant project) **Package:** `stmangsys` · **Main class:** `stmangsys.StMangSys` · **Java:** JDK 25 **UI:** Java Swing (NetBeans GUI Builder) · **Storage:** in-memory object arrays (data resets on exit, per spec)

---

1. System Overview

A desktop GUI application that manages students, batches and the two subject marks (PRF — Programming Fundamentals, DBMS — Database Management Systems) and generates reports. All requirements of the coursework specification are implemented:

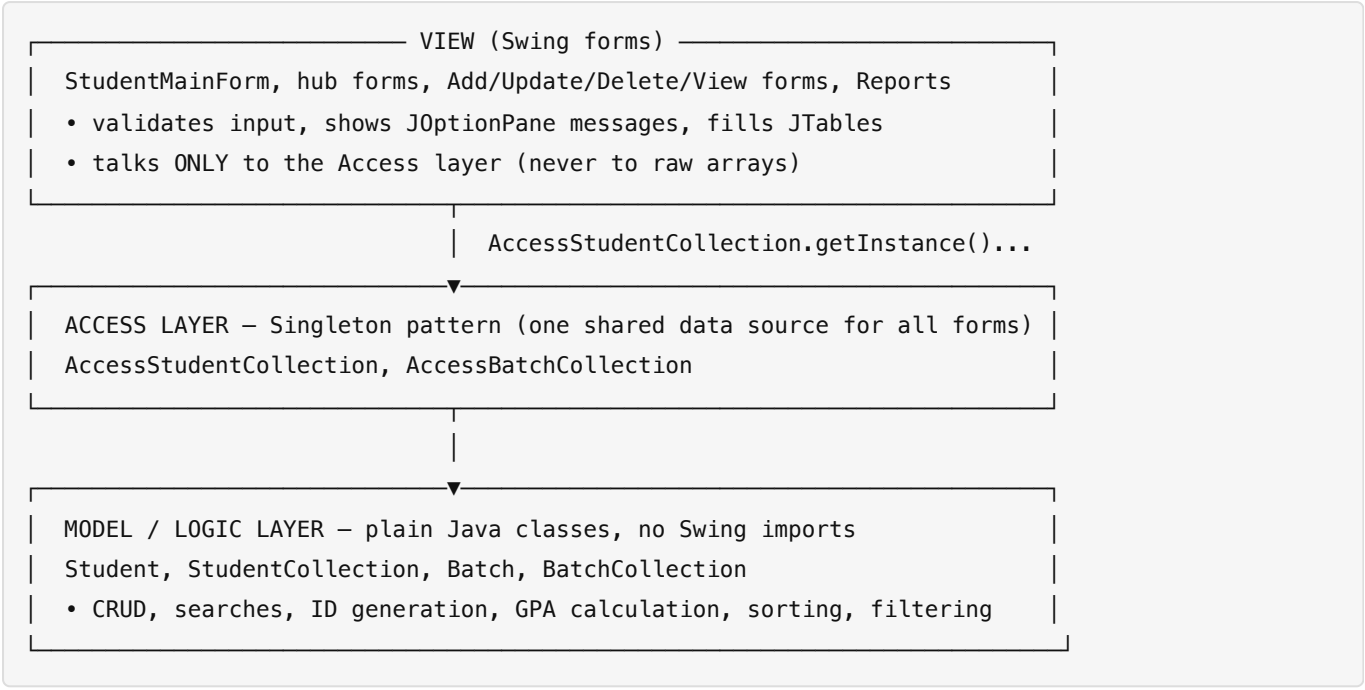
Spec	Feature	Where
1	Student Management (Add / Update / Delete / View / View All)	`ViewStudentsTables` + 5 forms
2	Batch Management (Add / Update status / View)	`BatchManagement` + 3 forms
3	Grade Management (PRF marks / DBMS marks update)	`GradeManagement` + 2 forms
4	Report Management (Registration / Batch-wise / Training Eligibility)	`ReportManagement` + 4 forms

Home page: `StudentMainForm` with the four menu buttons + Close.

---

2. Architecture

Three clean layers — this separation is the main design point to explain in the viva:



Benefits to state: the business logic is testable without the GUI, the GUI can be redesigned without touching logic, and there is exactly **one** copy of the data in memory (so a student added in one form is instantly visible in every other form).

---

3. Relevant Files (26 `.java` + 19 `.form` in `src/stmangsys/`)

Model / logic layer

File	Purpose
`Student.java`	Model: regNo, name, NIC, batchNo, prfMarks, dbmsMarks. Constants `MARK_ABSENT = -1`, `MARK_NOT_CONDUCTED = -2`. Holds the **GPA logic** (`marksGpa`, `getGpa`, `getGpaText`), display helper `marksText()`, and `isEligibleForIndustryTraining()`
`StudentCollection.java`	Array-backed CRUD: add (with regNo + NIC duplicate checks), `getAllStudent`, `getStudentsByBatch`, linear searches by regNo/NIC, `generateRegistrationNo()`, update / delete (array shifting), marks updates, and **name-sorted views** for reports (insertion sort)
`Batch.java`	Model: batchNo + status (`STATUS_OPEN = 1`, `STATUS_CLOSED = 0`), `getStatusText()`
`BatchCollection.java`	Array-backed batch CRUD, `generateNextBatchNo()` (starts at 100), `getStudentCount(batchNo)`

Access layer (Singleton)

File	Purpose
`AccessStudentCollection.java`	`getInstance()` returns one shared `StudentCollection`
`AccessBatchCollection.java`	Same pattern for batches

Entry point & home page

File	Purpose
`StMangSys.java`	`main()` : sets Nimbus look-and-feel, launches the main form on the Event Dispatch Thread
`StudentMainForm(.java/.form)`	Home page; buttons open each hub; Close = `System.exit(0)`

Student Management (spec 1)

File	Purpose
`ViewStudentsTables`	Hub (Add / Update / Search-View / Delete / View all)
`AddStudentForm`	Batch combo (only **open** batches; first open batch pre-selected), lecture-mode combo (Physical/Online), NIC duplicate check, **live auto reg-no preview**, closed-batch enrollment block
`UpdateStudentForm1`	Search by regNo → edit Name and NIC only (batch is fixed by spec)
`ViewStudent`	Read-only profile: regNo, name, NIC, PRF, DBMS, GPA
`DeleteStudent`	Search + confirm dialog → delete
`ViewStudentDetailsForm`	7-column JTable of all students (loads on construct)

Grade Management (spec 3)

File	Purpose
`GradeManagement`	Hub: PRF / DBMS buttons
`PrfMarksUpdateForm`, `DbmsMarksUpdateForm`	Search regNo → shows current details → validate new marks (`−1` = ABSENT with confirmation, else 0–100, non-numeric rejected) → update → success + clear

Batch Management (spec 2)

File	Purpose
`BatchManagement`	Hub
`AddBatchForm`	Auto-generated next batch no (read-only field), status combo
`UpdateBatchForm`	Search + change enrollment status
`ViewBatchesForm`	JTable: No, Batch No, Student Count, Status

Report Management (spec 4)

File	Purpose
`ReportManagement`	Hub with the 3 report buttons; opens a report and disposes itself so each report's "Back to Home Page" simply closes and reveals the home page
`StudentRegistrationReport`	**All** students sorted alphabetically by name
`SelectBatchForm`	Batch selection page — one dynamically created button per batch (e.g. "101 Batch")
`BatchWiseStudentReport(batchNo)`	Title "101 Batch Student Report"; only that batch, sorted by name
`TrainingEligibilityReport`	Filter: **GPA > 3.25 AND PRF > 50 AND DBMS > 50**

Every GUI form has a matching NetBeans `.form` XML file (see theory §4.1). Each report table has the same 5 columns: Batch No, Reg No, PRF Marks, DBMS Marks, GPA — marks shown as `ABSENT` / `NOT CONDUCTED` when applicable.

---

4. Theory Behind the Implementation

4.1 Java Swing & the NetBeans GUI Builder

- **Swing** is Java's GUI toolkit. `JFrame` is a *top-level container* (the window); the *content pane* holds child components (`JButton`, `JLabel`, `JTextField`, `JComboBox`, `JTable`, `JScrollPane`, `JPanel`).
- NetBeans GUI Builder stores the visual design in a **`.form` XML file** beside each class and regenerates the `initComponents()` code inside `// GEN-BEGIN/END` markers — user code outside those markers is preserved.
- Layouts: the forms use **`GroupLayout`** (the GUI Builder's layout, which aligns components into horizontal/vertical groups); `SelectBatchForm` switches a panel to **`BoxLayout` (PAGE_AXIS)** at runtime to stack one button per batch vertically.
- `pack()` sizes the window to the preferred sizes; `setLocationRelativeTo(null)` **centers** it on screen; `setResizable(false)` keeps the design fixed.
- `setDefaultCloseOperation`: `DISPOSE_ON_CLOSE` (child windows — releases the window, app keeps running) vs `EXIT_ON_CLOSE` (main form — ends the JVM).

4.2 Event-Driven Programming

- Swing is **event-driven**: the program waits for events on the **Event Dispatch Thread (EDT)**. Button clicks fire `ActionEvent`s delivered to registered `ActionListener`s (`addActionListener` → `actionPerformed(evt)`).
- `StMangSys.main()` uses `EventQueue.invokeLater(...)` so the GUI is created on the EDT — the thread-safe way to start Swing.
- **Dialogs** via `JOptionPane`: `showMessageDialog` (info/warnings), `showConfirmDialog` with `YES_NO_OPTION` (used for delete and for marking ABSENT).
- `JComboBox` selection changes also fire action events — used to refresh the live registration-number preview in `AddStudentForm`.

4.3 OOP Concepts Used

Concept	Where it appears
Encapsulation	All model fields are `private` with public getters/setters (`Student`, `Batch`) — state can only change through controlled methods
Abstraction / separation of concerns	Model classes have no Swing code; forms contain no data structures
Singleton pattern	`AccessStudentCollection` / `AccessBatchCollection`: `private` constructor + `static getInstance()` with lazy creation → **one shared instance** of the data for all forms
Composition (has-a)	`StudentCollection` *has an* array of `Student`; `AccessStudentCollection` *has a* `StudentCollection`
Class constants	`MARK_ABSENT/-1`, `MARK_NOT_CONDUCTED/-2`, `STATUS_OPEN/1`, `STATUS_CLOSED/0` — magic numbers avoided
Constructors & overloading	Each form initializes UI in its constructor; `BatchWiseStudentReport` has a parameterized constructor taking the batch number
Method overriding	`toString()` overridden in `Student` and `Batch` for console-style output

4.4 Data Structures & Algorithms (coursework style — plain arrays)

- **Object arrays** (`Student[]`, `Batch[]`) instead of `ArrayList`, matching the coursework's array-based style.
- **Dynamic growth**: `extendsStudentArray()` creates a bigger array and copies elements (arrays in Java are fixed-size).
- **Linear search** (`searchStudent`, `searchStudentByNic`) — O(n), case-insensitive via `equalsIgnoreCase`.
- **Deletion by shifting**: `deleteStudent` copies all elements except the target into a smaller array.
- **Defensive copies**: `getAllStudent()` returns a *copy* of the array so callers cannot mutate the collection's internal state.
- **Insertion sort** (`sortByName`) orders reports alphabetically using `compareToIgnoreCase` — O(n^2) but simple and ideal for small data sets.
- **Registration-number generation** scans the batch for the highest 3-digit sequence and adds 1 → `PR/OR + 24 + batchNo + 001…` (e.g. `PR24105003`). The sequence is **batch-wide** regardless of PR/OR prefix.

4.5 Business Rules

- A student's **reg no is system-generated** (never typed); NIC and regNo must be unique.
- Students can only be **added to OPEN batches**; closed batches accept no enrollments.
- **Marks conventions**: `-1` = ABSENT, `-2` = NOT CONDUCTED (default on add). Any other value must be 0–100.
- **Student GPA = (PRF GPA + DBMS GPA) ÷ 2**, using the spec's GPA value table. A subject without a real mark contributes **0** to the average — this reproduces the spec's worked sample exactly: PRF 94 → 4.25, DBMS 39 → 1.00 → (4.25 + 1.00)/2 = **2.625**.
- **Industry Training Eligibility** (spec 4.3): GPA **> 3.25** AND **both** PRF and DBMS marks **> 50**.

GPA value table (Figure 12 of the spec):

Marks	GPA	Marks	GPA
90–100	4.25	50–54	2.00
80–89	4.00	45–49	1.70
75–79	3.70	40–44	1.30
70–74	3.30	30–39	1.00
65–69	3.00	0–29	0.70
60–64	2.70	absent / not conducted	counts as 0
55–59	2.30		

Implemented as an if/else-if ladder in `Student.marksGpa(int)` — highest range checked first, so a single pass decides the band.

4.6 Report Generation & `JTable`

- Reports are **views over the same in-memory data**: each form asks the singleton for a fresh copy, sorts it (name order) and optionally filters it (batch no / eligibility) — so reports always reflect the latest adds, edits and marks.
- `JTable` + **`DefaultTableModel`**: the model holds rows of data (`setRowCount(0)` then `addRow(...)`), the `JTable` renders it; `JScrollPane` adds scroll bars automatically.
- Marked values are displayed via `Student.marksText()`; GPA via `getGpaText()` (prints whole numbers without decimals, e.g. `4`).

---

5. Key Code Walkthroughs (be ready to explain these)

GPA band lookup (`Student.marksGpa`) — guard clause + ladder:

```
public static double marksGpa(int marks) {
    if (!isRealMark(marks)) return -1;      // absent / not conducted
    if (marks >= 90) return 4.25;
    else if (marks >= 80) return 4.00;
    // ... down to 0.70 for 0–29
}
public double getGpa() {                    // (PRF GPA + DBMS GPA) / 2
    double prfGpa  = Math.max(marksGpa(prfMarks), 0);   // no-GPA subject counts 0
    double dbmsGpa = Math.max(marksGpa(dbmsMarks), 0);
    return (prfGpa + dbmsGpa) / 2.0;
}
```

Singleton (`AccessStudentCollection`) — lazy creation, one instance:

```
private static AccessStudentCollection accessStudentCollection;
public static AccessStudentCollection getInstance() {
    if (accessStudentCollection == null)
        accessStudentCollection = new AccessStudentCollection();
    return accessStudentCollection;
}
```

Registration number generation (`StudentCollection.generateRegistrationNo`)

```
String prefix = physical ? "PR" : "OR";          // lecture mode
String year   = "24";                            // 2024 intake
// scan batch for the highest existing 3-digit sequence, then highest + 1,
// zero-padded: PR + 24 + 105 + 003  ->  PR24105003
```

Insertion sort for report ordering (`sortByName`)

```
for (int i = 1; i < students.length; i++) {
    Student key = students[i];
    int j = i - 1;
    while (j >= 0 && students[j].getName().compareToIgnoreCase(key.getName()) > 0) {
        students[j + 1] = students[j];
        j--;
    }
    students[j + 1] = key;
}
```

Eligibility rule (`Student.isEligibleForIndustryTraining`)

```
return getGpa() > 3.25
        && isRealMark(prfMarks) && prfMarks > 50
        && isRealMark(dbmsMarks) && dbmsMarks > 50;
```

Add-student validation chain (`AddStudentForm`) — in order: batch selected → batch exists → **enrollment OPEN** → NIC not empty → name not empty → **NIC not already registered** (shows the existing student's profile if it is) → generate regNo → add.

---

6. Worked Examples (use these numbers in the viva)

Student	PRF	DBMS	PRF GPA	DBMS GPA	Student GPA	Training eligible?
Spec sample	94	39	4.25	1.00	(4.25+1.00)/2 = **2.625**	No — DBMS ≤ 50
Bimal Silva	85	80	4.00	4.00	**4.0**	**Yes**
Sunil Rathnayake	70	70	3.30	3.30	**3.3**	**Yes** (3.3 > 3.25)
Nirosha Jayasinghe	65	65	3.00	3.00	**3.0**	No — GPA ≤ 3.25 despite marks > 50
Not conducted	–2	–2	0	0	**0**	No

---

7. Verification Evidence

The whole application was driven automatically like a user (a Swing test driver that clicks every button, enters data, answers dialogs and screenshots every screen):

- **32/32 automated UI checks pass**, covering: add/update/delete/view flows, invalid reg no, empty-field and out-of-range (150) marks validation, ABSENT confirmation, closed-batch enrollment block, NIC duplicate check, auto reg-no `PR24101004` issued, GPA 2.5 and the spec sample 2.625, batch add/update, and all three reports (alphabetical order, batch filtering, eligibility filtering).
- 46 screenshots in `/tmp/walk/` (gallery `index.html`).
- All 19 `.form` XML files validated; project compiles cleanly on JDK 25.

Demo order for the viva: Home → Student Management (add → view all table shows the new student) → Grade Management (enter 94/39 → profile shows 2.625) → Batch Management (103 added, then closed) → Report Generator (three reports) → Close.

---

8. Likely Viva Questions & Short Answers

1. **Why a Singleton for the collections?** All forms must share the *same* data. Without it, each form would create its own `StudentCollection` and a student added in one window would not appear in another.
2. **Why arrays instead of `ArrayList`?** Coursework requirement/style; it also demonstrates manual array growth, copying and shifting, which `ArrayList` hides.
3. **Where is data stored?** In memory only — arrays inside the singletons. Closing the app discards data (as the spec allows; a file/DB layer could be future work).
4. **What is `initComponents()`?** NetBeans GUI-Builder-generated code that creates and lays out every component; it lives inside `GEN` markers and must not be edited.
5. **What is the `.form` file?** XML design file the GUI Builder uses to regenerate `initComponents()`; deleting it breaks the two-way editing (the `.java` still compiles).
6. **Why `GroupLayout`?** It's what the GUI Builder generates — it defines independent horizontal and vertical groups, giving precise alignment and resizable gaps.
7. **Difference between `DISPOSE_ON_CLOSE` and `EXIT_ON_CLOSE`?** Dispose releases just that window (used on all child forms); Exit terminates the JVM (used on the main form, plus the Close button calls

`System.exit(0)`).

8. **Which thread runs button handlers?** The Event Dispatch Thread (EDT). That's why `main()` starts the GUI inside `EventQueue.invokeLater`.
9. **How is the registration number generated?** Prefix (PR physical / OR online) + intake year `24` + batch number + zero-padded 3-digit sequence = highest sequence in that batch + 1, e.g. `PR24101004`.
10. **Why is the sequence batch-wide, not per prefix?** The spec's sample IDs run sequentially per batch (…001, …002, …003) regardless of lecture mode, so PR and OR students of one batch share one counter.
11. **How does the closed-batch rule work?** `AddStudentForm` checks `batch.getStatus() == Batch.STATUS_CLOSED` before adding and blocks with a message; the batch combo also pre-selects the first open batch for convenience.
12. **What happens to a subject with no marks in GPA?** `marksGpa` returns –1 for non-real marks and `getGpa()` clamps it to 0, so the average is computed as if that subject scored 0 — matching the spec's sample (94 + 39 → 2.625).
13. **Why insertion sort rather than `Arrays.sort`?** Small data sets; it is simple, stable, written in the coursework's plain-array style, and easy to explain.
14. **How does the batch-wise report get its batch?** `SelectBatchForm` creates one button per batch and sets the batch number as the button's **action command**; the listener passes it to the `BatchWiseStudentReport(batchNo)` constructor.
15. **What is `DefaultTableModel`?** The data holder behind `JTable`; the report code clears it (`setRowCount(0)`) and fills rows with `addRow(...)` on every load, so the table always shows current data.
16. **How did you test it?** An automated driver walked every screen like a user — clicking buttons, typing data, answering dialogs — asserting 32 behaviours (validations, GPA values, ID generation, report contents) and capturing a screenshot of every step.

---

9. How to Run

- **NetBeans:** open the `StMangSys` project, set `stmangsys.StMangSys` as main class, Run (F6).
- **Command line:**

```
javac -d build/classes $(find src -name "*.java")
java -cp build/classes stmangsys.StMangSys
```

- Seeded demo data: batches 100 (CLOSED), 101, 102 (OPEN); students Nimal Perera, Bimal Silva, Amal Fernando (batch 101), Nirosha Jayasinghe (batch 102) — all marks initially NOT CONDUCTED.